

Veil: A Backend-Agnostic Protocol for Confidential Token Distribution on Public Blockchains

Veil Protocol

Zama Developer Program, Season 3 — Special Bounty Track × TokenOps

Sepolia: ConfidentialDisperse 0x86E0497dFD8FDbFB25F6AE3DC8104c46bBE8329 · v0.2

Abstract— Token distributions on public blockchains—airdrops, payroll, investor distributions, vesting—leak two sensitive dimensions: *how much* each recipient receives and *who* the recipients are. Existing confidential-token schemes based on fully homomorphic encryption (FHE) hide amounts but leave recipient addresses, and therefore the distribution graph, fully public. We present **Veil**, a protocol that composes FHE-based confidential tokens (ERC-7984) with stealth-address cryptography (ERC-5564 / ERC-6538) to hide both dimensions simultaneously. An onchain observer learns only that a distribution occurred and an upper bound on its size; recipients independently scan, detect, and decrypt *only their own allocation* through an EIP-712 user-decryption flow. We formalize the protocol, analyze its security by reduction to the hardness of the underlying primitives, and explicitly characterize its residual leakage. We report a working deployment on the Sepolia testnet in which a single disperse transaction pays two stealth recipients different encrypted amounts, each decrypting only their own. Finally, we abstract the confidential-compute backend behind a narrow interface, positioning Veil as an application-layer payments protocol rather than a compute network.

1. Introduction

Public blockchains are radically transparent by design. That transparency is an asset for auditability and a liability for finance. When an organization runs payroll onchain, every salary is public. When a project runs an airdrop, the full recipient list and per-recipient amounts are permanently legible to competitors, attackers, and the recipients' own counterparties.

FHE-based confidential tokens (ERC-7984, enabled by the Zama Protocol) solve *half* of this: balances and transfer amounts become ciphertexts (`euint64`) that only authorized parties can decrypt. But the **recipient address remains in the clear**. An observer who cannot read amounts can still reconstruct the *shape* of a distribution—who received something, how many recipients there were, and how the recipients relate to other onchain identities. For payroll and airdrops, the recipient graph is often as sensitive as the amounts.

Veil closes the gap by routing each confidential transfer to a **one-time stealth address** unlinkable to the recipient's public identity, and announcing it through the ERC-5564 standard so the recipient—and only the recipient—can detect it. The result: amounts hidden by FHE, identities hidden by stealth addresses, and a clean self-service claim flow.

Contributions.

- A protocol composing ERC-7984 confidential tokens with ERC-5564/6538 stealth addresses for fully confidential disperse (§5).
- A security analysis reducing each privacy property to a standard assumption, plus an honest leakage characterization and a decoy-padding mitigation (§6).
- A backend-abstraction argument showing the confidential-compute layer (FHE today, MPC tomorrow) is replaceable behind a narrow interface (§7).
- A working Sepolia deployment with an end-to-end confidential disperse and self-service claim (§8).

2. Threat Model

Parties. A *sender* (distribution organizer), a set of *recipients*, and the *Zama coprocessor* / *threshold decryption network*. An optional *relayer* submits transactions on the sender's behalf.

Adversary. A computationally bounded observer with full read access to chain state and the public mempool. The adversary may control some recipients and may collude with a relayer. We assume the threshold decryption committee is honest below its corruption threshold (inherited from the Zama Protocol).

Goals (in scope). *Amount confidentiality*: the adversary learns nothing about any honest recipient's amount. *Recipient unlinkability*: the adversary cannot link a stealth address to a recipient's published meta-address or any other onchain identity.

Out of scope (stated honestly; mitigations in §6): the recipient *count* (or a padded upper bound); sender identity (the sender signs the disperse transaction); network-level metadata (timing, IP, gas-pattern side channels).

3. Background

3.1 FHE and confidential tokens (ERC-7984)

Fully homomorphic encryption permits computation directly on ciphertexts. The Zama Protocol exposes FHE to EVM contracts through a coprocessor: contracts manipulate ciphertext *handles* (`euint64`) while the actual TFHE operations execute off-chain and results are attested back. ERC-7984 is the confidential fungible-token standard: balances are encrypted, and access to decrypt a given ciphertext is governed by an onchain **access-control list (ACL)** managed via `FHE.allow`.

3.2 User decryption (EIP-712)

A user reads their own ciphertext by signing an EIP-712 request authorizing the threshold network to re-encrypt the value under the user's key. Only a party the ACL has granted access to can obtain a valid decryption.

3.3 Stealth addresses (ERC-5564 / ERC-6538)

A recipient publishes a **meta-address** $M = (P_{\text{spend}}, P_{\text{view}})$ to the ERC-6538 registry. A sender derives a fresh, single-use **stealth address** using Diffie–Hellman against P_{view} , and posts an **announcement** (ERC-5564) containing an ephemeral public key and a one-byte *view tag*. The recipient scans announcements with their viewing key; the view tag lets them reject $\sim 255/256$ of irrelevant announcements before doing elliptic-curve work.

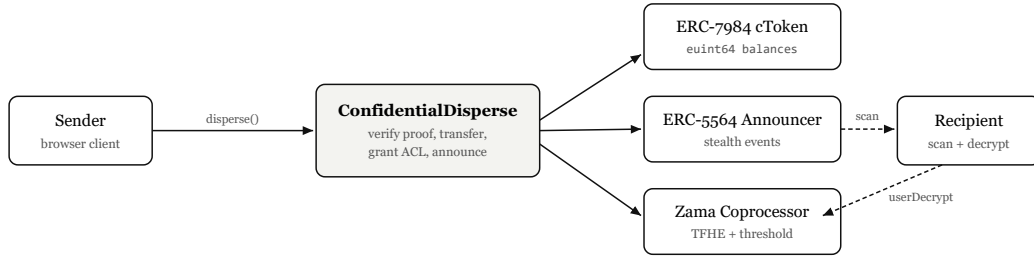


Figure 1. System architecture. Solid arrows: the sender's single disperse transaction. Dashed: the recipient's independent scan and EIP-712 user-decryption. The contract holds no funds.

4. Notation

Symbol	Meaning
G, n	secp256k1 generator and group order
$H(\cdot)$	hash to scalar mod n
$M_i = (P_{\text{spend}}, P_{\text{view}})$	recipient i meta-address (public)
$(p_{\text{spend}}, p_{\text{view}})$	recipient i private keys
$(r_i, R_i = r_i \cdot G)$	sender ephemeral keypair
s_i	ECDH shared secret
A_i	stealth address for recipient i
v_i	one-byte view tag
ct_i	FHE ciphertext of amount

5. The Veil Protocol

5.1 Registration (recipient, once)

Recipient i publishes $M_i = (P_{\text{spend}}, P_{\text{view}})$ to the ERC-6538 registry.

5.2 Disperse (sender)

For each recipient i with amount m_i , the client derives:

```

1.  $(r_i, R_i = r_i \cdot G)$  // fresh ephemeral keypair
2.  $s_i = H(r_i \cdot P_{\text{view}})$  // ECDH against viewing key
3.  $A_i = \text{addr}(P_{\text{spend}} + H(s_i) \cdot G)$  // one-time stealth address
4.  $v_i = s_i[0]$  // 1-byte view tag
5.  $ct_i = \text{FHE.encrypt}(m_i) \rightarrow (h_i, \pi)$  // client-side;  $\pi$  = ZK input proof
6. submit  $\text{disperse}(A[], h[], \pi, R[], v[])$ 
```

Onchain, for each i , the contract executes:

```

amt_i = FHE.fromExternal(h_i,  $\pi$ ) // verify input proof
FHE.allowTransient(amt_i, cToken) // token may operate on handle
cToken.confidentialTransferFrom(sender, A_i, amt_i)
announcer.announce(SCHEME_ID, A_i, R_i, v_i)
```

The ERC-7984 token's internal `_update` grants the recipient decrypt rights on both the transferred amount and the new balance (`FHE.allow(·, A_i)`). This is essential: the stealth address is newly derived and has no prior ACL entry, so without the grant the recipient could never decrypt the received balance. The sender authorizes the contract once via `cToken.setOperator`.

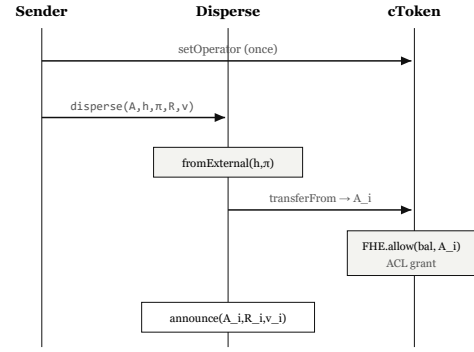


Figure 2. Disperse write path (per recipient, within one transaction). The token, not the Veil contract, performs the ACL grant to the freshly derived stealth address.

5.3 Scan & Claim (recipient)

```

for each Announcement (A, R, v):
  s = H( p_view · R ) // ECDH against ephemeral key
  if s[0] != v: continue // view-tag fast reject (~255/256)
  if addr(P_spend + H(s)·G) == A: // it's mine
    p = (p_spend + H(s)) mod n // stealth private key
    balance = userDecrypt(cToken, A) // EIP-712, signed by p
```



Figure 3. Scan funnel. The one-byte view tag discards $\sim 255/256$ of announcements before any elliptic-curve work; only a full key match triggers an EIP-712 decryption.

6. Security Analysis

6.1 Amount confidentiality

Amounts are encrypted under TFHE and only decryptable by parties the ACL authorizes. An adversary distinguishing two amounts breaks TFHE IND-CPA or the threshold-decryption security of the Zama network. Honest recipients' amounts are therefore semantically hidden from any observer not granted ACL access.

6.2 Recipient unlinkability

Linking A_i to M_i requires computing $s_i = H(r_i \cdot P_{\text{view}})$ from public values R_i and P_{view} —i.e., solving computational Diffie–Hellman on secp256k1. Under the DDH/CDH assumption, stealth addresses are unlinkable to meta-addresses and to each other. This is the standard ERC-5564 guarantee.

6.3 Combined view

An observer with only chain data learns: (a) that a disperse transaction occurred, (b) the sender, and (c) the number of announcements. They learn neither amounts nor recipient identities.

6.4 Residual leakage and mitigations

Table 1. Residual leakage under the §2 adversary, with mitigations.

Leak	Sev.	Mitigation
Recipient count	med	Decoy padding: dummy announcements to a fixed N; decoys carry <code>FHE.encrypt(o)</code> , indistinguishable onchain.
Sender identity	med	Relayer / meta-transaction / account-abstraction sponsor.
Timing, gas	low	Fixed batch size + uniform per-recipient operations.
Total amount	none	FHE hides the homomorphic sum if exposed at all.

Disclosing leakage explicitly is a feature, not a weakness: a protocol that names its boundaries is auditable; one that claims “fully private” is not.

7. Backend Abstraction

The protocol uses the confidential-compute backend only to (i) encrypt an amount to a ciphertext handle with a verifiable input proof, (ii) transfer the handle under an ACL, and (iii) authorize a per-recipient decryption. This is a narrow interface:

```
ConfidentialBackend:
  encrypt(m)      -> (handle, proof)
  transfer(from, to, h) -> ()      // ACL-gated
  grant(h, addr)  -> ()          // extend ACL
  userDecrypt(addr, h) -> m       // authorized only
```

FHE (Zama / ERC-7984) is the instantiation used here. An MPC backend (secret-shared amounts with a committee performing the transfer and selective reveal) satisfies the same interface. Veil is therefore an **application-layer payments protocol**, not a compute network: it composes with whatever confidential-compute substrate is available. This is the axis on which Veil is complementary to—not competing with—general MPC networks such as Arcium.

8. Implementation & Deployment

Contracts. `ConfidentialDisperse.sol` (Solidity, FHEVM) reuses the canonical ERC-5564 announcer and operates over the official ERC-7984 cToken from the Zama Wrappers Registry, integrating the TokenOps SDK for the distribution primitive. **Client.** TypeScript with `@zama-fhe/relayer-sdk` for input encryption and EIP-712 user decryption, and `@noble/curves` for ECDH and stealth-key derivation. The contract carries no FHE state and no funds custody (pull-then-push within one transaction).

Live deployment (Sepolia). The protocol is deployed and verified end-to-end. In a representative run, a single `disperse` transaction paid two independent stealth recipients encrypted amounts of 300 and 700 cUSDt; each recipient, holding only its stealth key, decrypted *only its own* allocation through the EIP-712 flow, and a cross-decryption attempt by the other recipient failed as expected.

Table 2. Sepolia deployment artifacts (verifiable onchain).

Artifact	Address / hash
ConfidentialDisperse	0xB6E0497dF08FDbfFB25F6AE3DC8104c46bBE8329
ERC-7984 cUSDt (official)	0x4E7B06D78965594eB5EF5414c357ca21E1554491
ERC-5564 Announcer	0x55649E01B5Df198D18D95b5c5c051630cfD45564
Disperse→claim tx	0x1e19e6a404a403fb4f39090ac954480e06e6ceb752119e3

9. Evaluation

We characterize three quantities: (i) gas per recipient for `disperse` as a function of batch size; (ii) recipient scan time versus announcement volume, with and without view tags; and (iii) FHE batch limits, which bound the maximum recipients per transaction. At demo scale (5–10 recipients) a single transaction suffices; the view tag reduces recipient-side elliptic-curve work by a factor of ~256. A full quantitative evaluation is deferred to an extended version.

10. Related Work

Combining FHE with stealth addresses is not new. HE-DKSAP and FHE-DKSAP (Dual-Key Stealth Address Protocols via (fully) homomorphic encryption, 2023) apply homomorphic encryption to the *stealth-address key exchange itself*, hardening recipient-address derivation. Veil differs in purpose and architecture: it applies FHE to the **transfer amount** (via ERC-7984) and uses standard ECDH stealth addresses (ERC-5564) for the **recipient**, rather than encrypting the key exchange. Veil’s contribution is *applied and systems-level*—a deployed confidential-distribution protocol with a self-service claim flow—not a new primitive.

Other related work: Umbra (stealth payments, amounts public); Tornado Cash / Privacy Pools (mixers, fungible-set anonymity); Zcash and Aztec (shielded pools); Fairblock and threshold-encryption mempools; Arcium (general MPC compute network, a *backend* in our \$7 abstraction rather than a competitor).

11. Limitations & Future Work

Count leakage absent decoy padding; sender anonymity requires relayers; view-tag scanning cost grows with announcement volume (amortizable via indexers); a formal game-based proof of the combined construction is left to an extended version.

12. Conclusion

Veil shows that hiding *who* and *how much* in onchain distributions is achievable today by composing two mature standards, and demonstrates it with a live Sepolia deployment. Its backend abstraction positions confidential distribution as a portable protocol rather than a property of any single cryptographic substrate.

13. References

[1] ERC-7984: Confidential Fungible Token Standard. [2] ERC-5564: Stealth Addresses. [3] ERC-6538: Stealth Meta-Address Registry. [4] EIP-712: Typed Structured Data Hashing and Signing. [5] I. Chillotti et al. TFHE: Fast Fully Homomorphic Encryption over the Torus. J. Cryptology, 2020. [6] Zama Protocol Litepaper, 2024–2026. [7] HE-DKSAP / FHE-DKSAP: Dual-Key Stealth Address Protocols via Homomorphic Encryption, 2023. [8] ScopeLift. Umbra: Stealth Payments on Ethereum. [9] Arcium: Confidential Computing Network. Documentation.